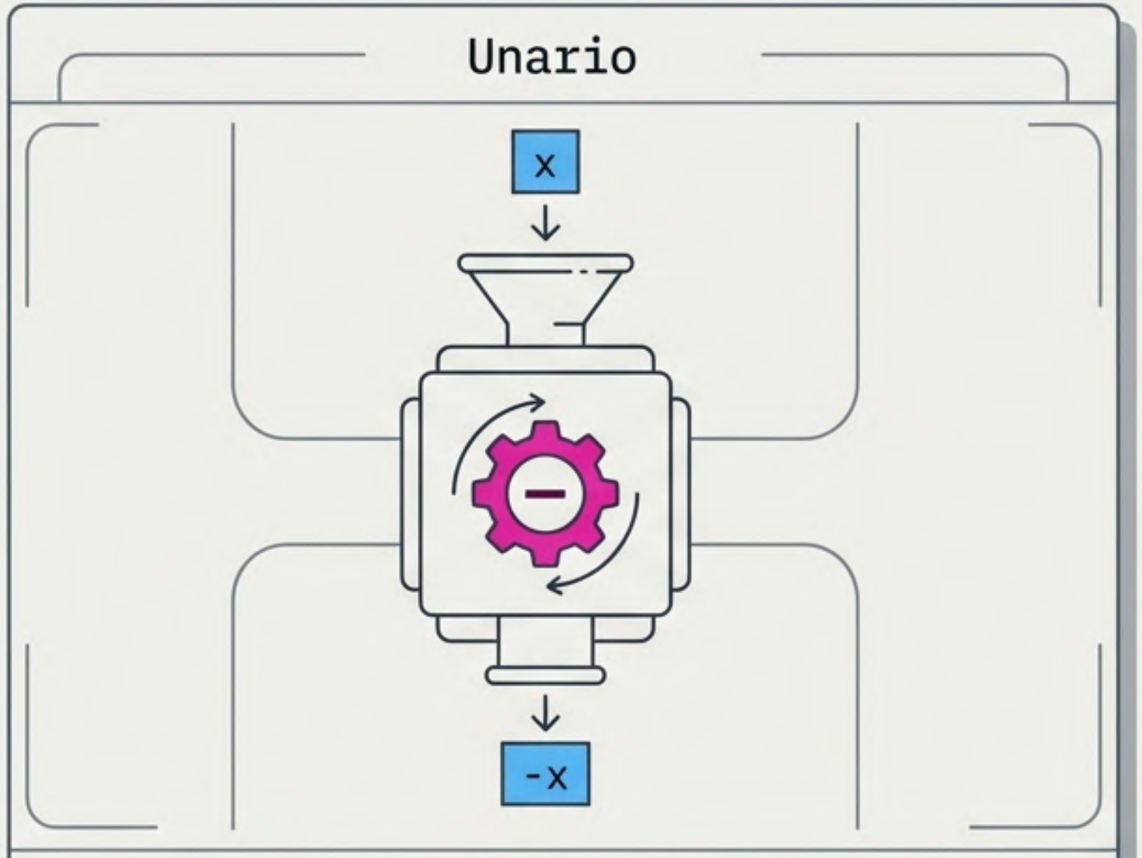
# JavaScript Core: Operadores, Lógica y Control de Flujo

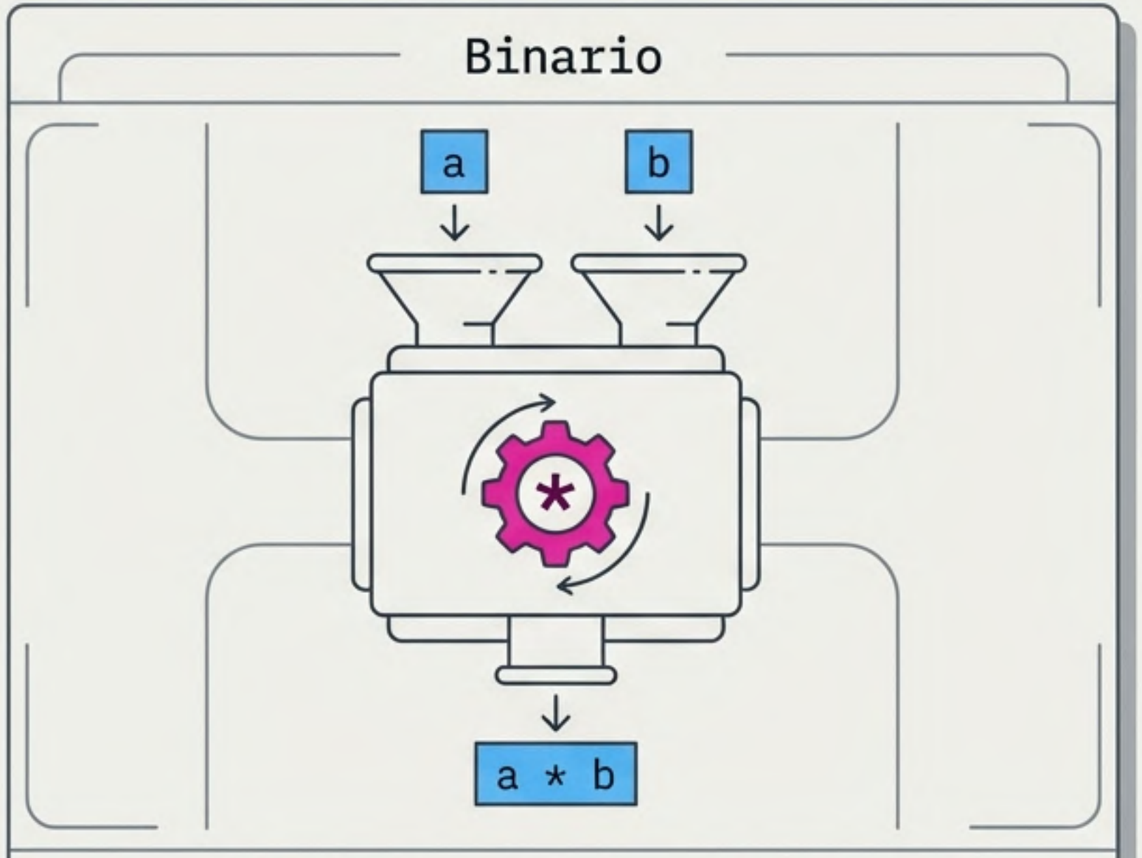
El manual visual definitivo sobre cómo el motor de JS lee, evalúa y decide.

NotebookLM

# Anatomía de una Expresión: Unario vs. Binario




**Unario:** Requiere un solo operando.



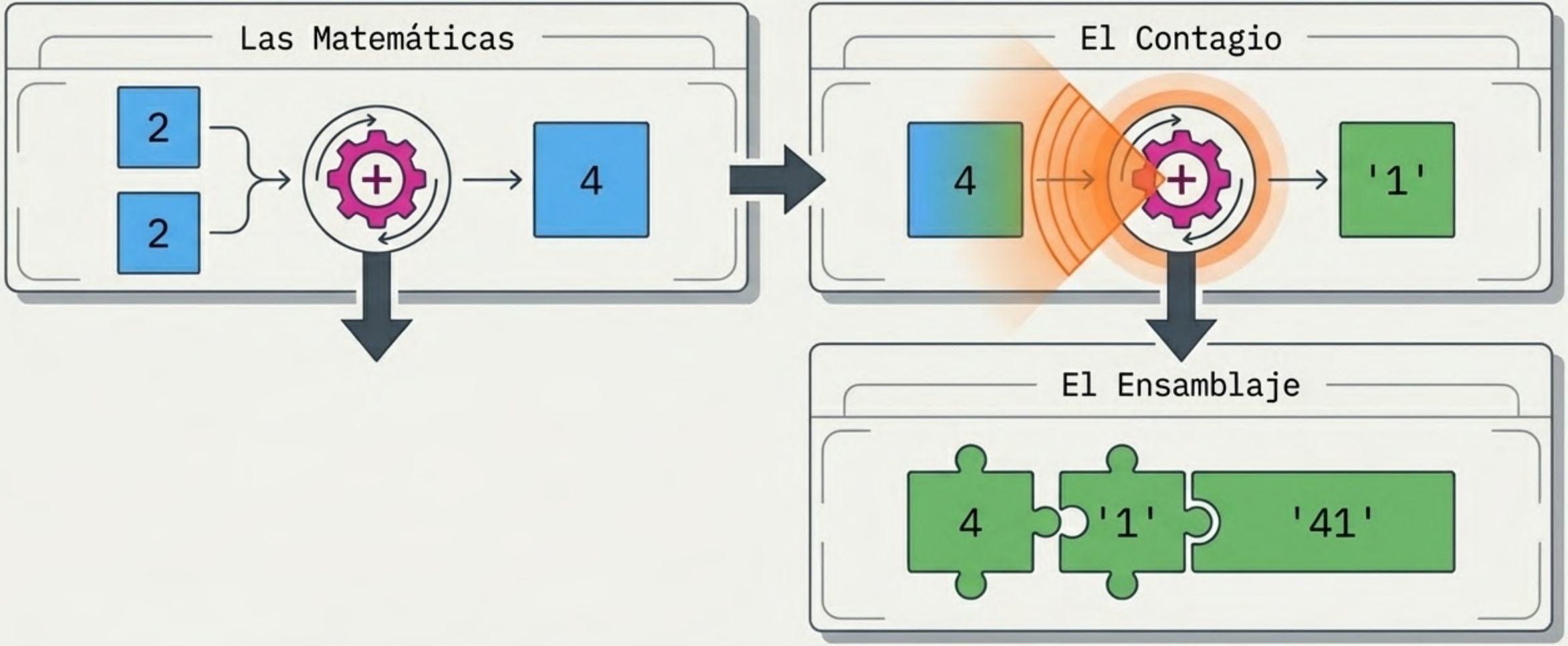

**Binario:** Requiere dos operandos.

Un **operando** es el dato. Un **operador** es la máquina.
El mismo símbolo puede actuar como ambas máquinas dependiendo del contexto.

NotebookLM

# La Doble Vida del Operador +

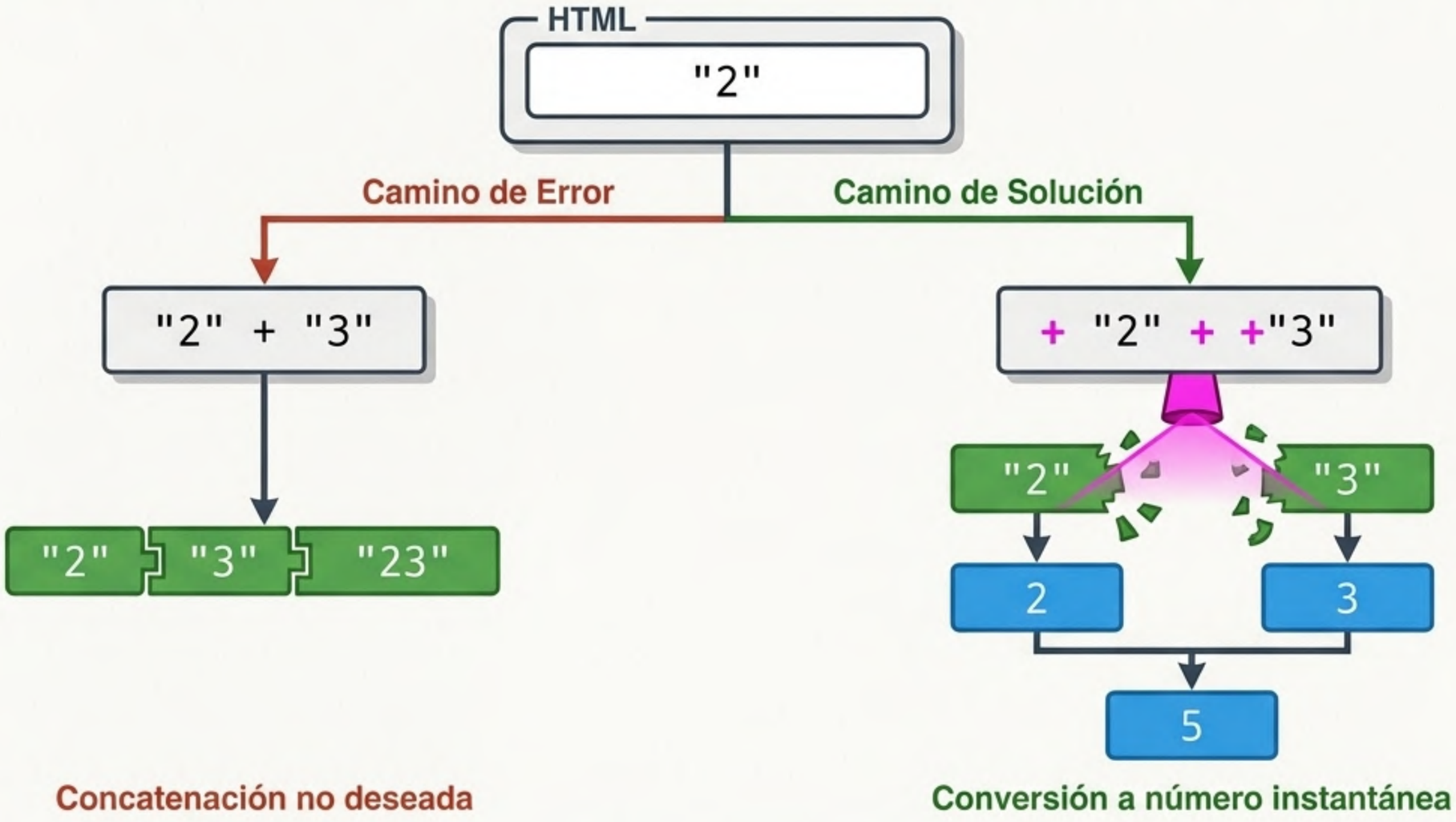
```
2 + 2 + '1' = '41'
```




Si algún operando es un String, el operador binario + detiene las matemáticas.
Convierte todo lo demás en texto y los concatena uno tras otro.

NotebookLM

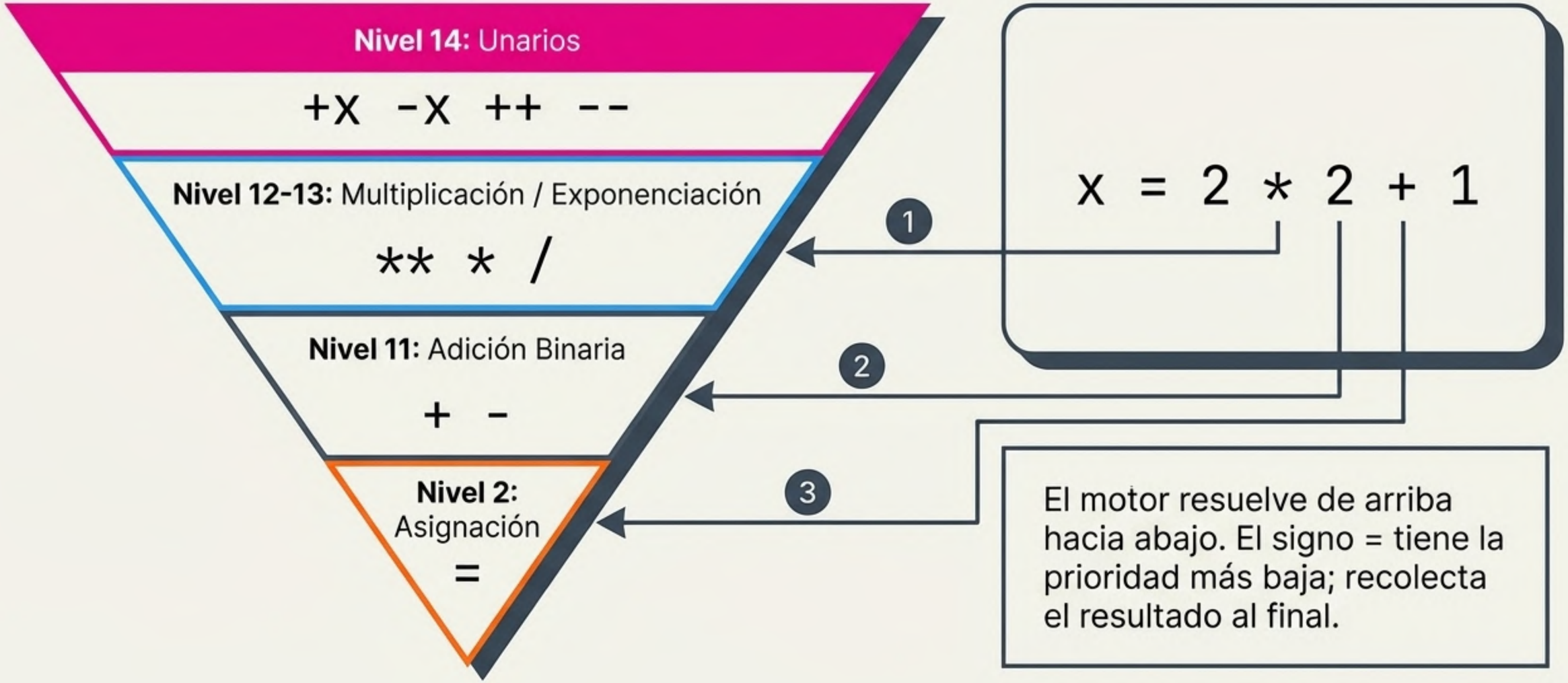
# El Poder del + Unario (Conversión Rápida)




El operador `+` unario aplicado a un `String` funciona de manera idéntica a `Number()`, pero de forma mucho más compacta.

NotebookLM

# La Pirámide de Jerarquía (Precedencia)




El motor resuelve de arriba hacia abajo. El signo = tiene la prioridad más baja; recolecta el resultado al final.

NotebookLM

# El Misterio del ++: Prefijo vs. Sufijo

**Prefijo: ++a**

```
let a = 1;
alert(++a); // Muestra 2
```

**Paso 1**: Suma 1 a la memoria (a=2).

**Paso 2**: Devuelve el NUEVO valor (2).

**Sufijo: b++**

```
let b = 1;
alert(b++); // Muestra 1
```

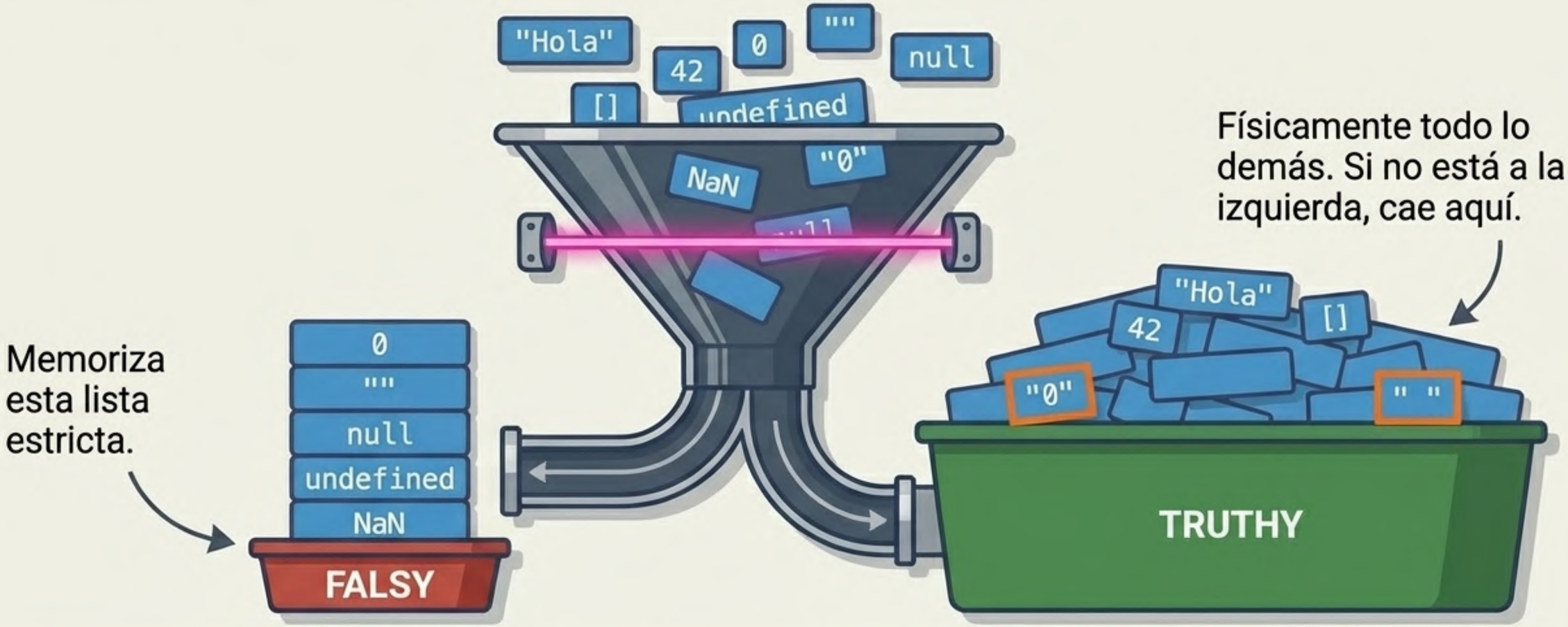
**Paso 1**: Devuelve el VIEJO valor (1).

**Paso 2**: Suma 1 a la memoria de fondo (b=2).

Ambos **modifican** la variable original. La diferencia crítica es el **orden de los eventos**: ¿se entrega el valor **antes** o **después** de la actualización?

NotebookLM

# El Embudo Booleano: Truthy vs. Falsy




En contextos lógicos (como un `if`), JavaScript convierte cualquier valor a Booleano usando este mecanismo de clasificación física.

NotebookLM

# Comparaciones y el Orden Lexicográfico

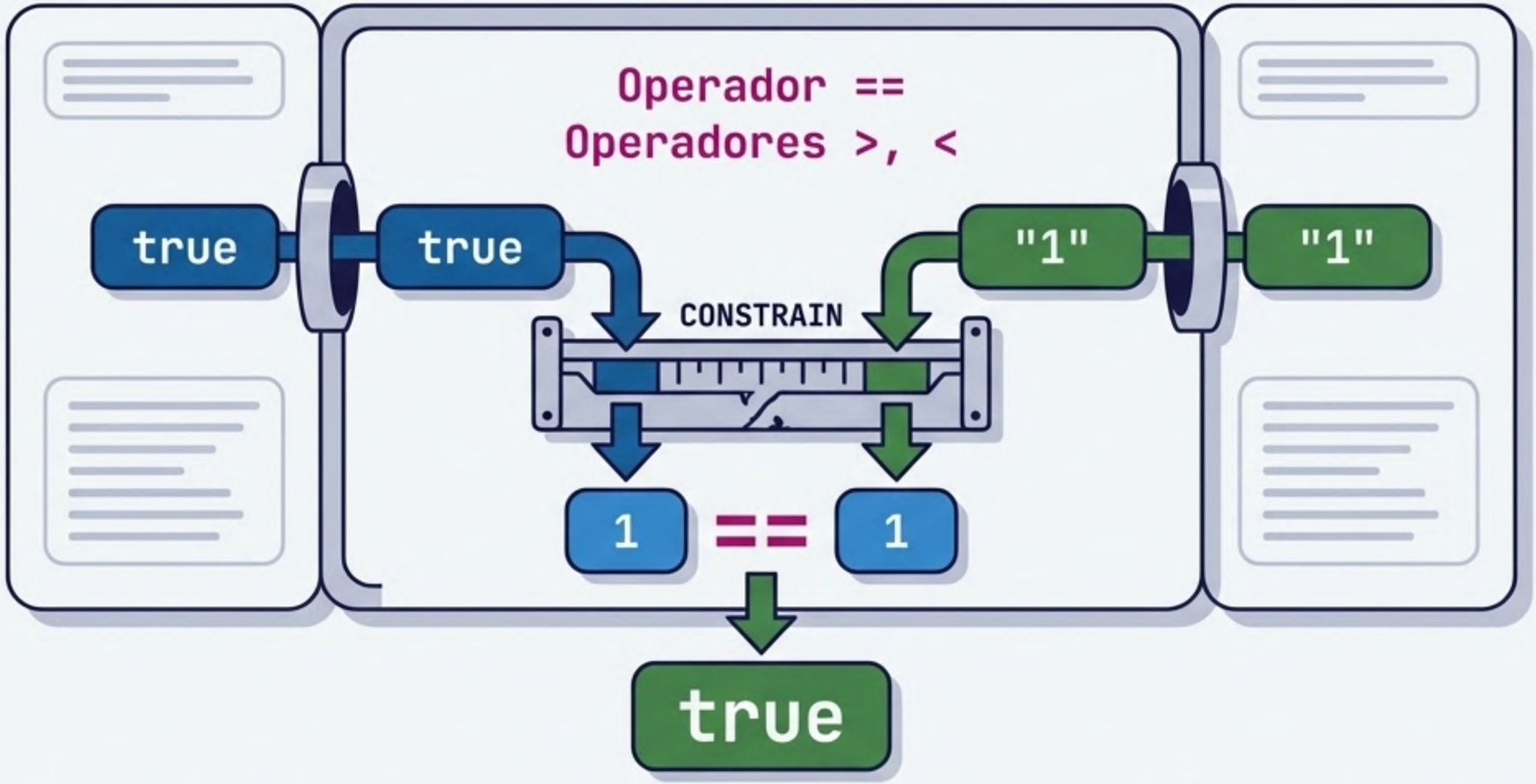
### Sensibilidad de Mayúsculas

`'a' > 'A'` → `true`

JavaScript no usa un diccionario tradicional, sino el índice de codificación Unicode. Las minúsculas tienen un índice mayor que las mayúsculas.

NotebookLM

# El Choque de Tipos: Conversión Numérica Oculta




Al usar comparadores relacionales (>, <) o igualdad débil (==), JavaScript no tolera tipos mixtos.

Fuerza a ambos contendientes a convertirse en números puros antes de evaluar, lo que causa falsos positivos.

NotebookLM

# Matriz Diagnóstica: == vs ===

| Comparación | == (Igualdad Débil) | === (Igualdad Estricta) |
|---|---|---|
| “0” vs 0 | ⓘ **true** | **false** |
| false vs 0 | ⓘ **true** | **false** |
| “” vs 0 | ⓘ **true** | **false** |
| “ ” vs 0 | ⓘ **true** | **false** |

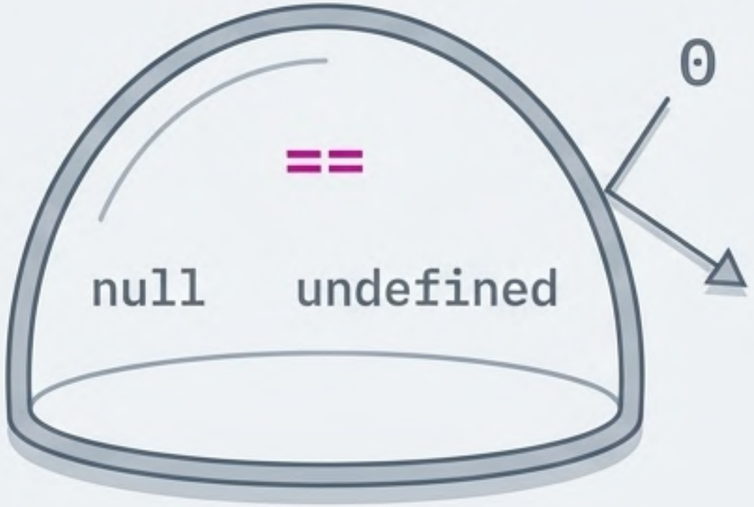
## El estándar de oro

El operador == convierte tipos tras bambalinas, provocando falsos positivos peligrosos.
El operador === (Igualdad Estricta) verifica primero el tipo de dato sin realizar conversiones. Úsalo siempre para prevenir bugs.

NotebookLM

# La Paradoja de null y undefined

## La Pareja Exclusiva




Regla Especial: `null` y `undefined` solo son iguales entre sí bajo ==. Ignoran cualquier conversión numérica y rechazan al 0.

## La Trampa Matemática

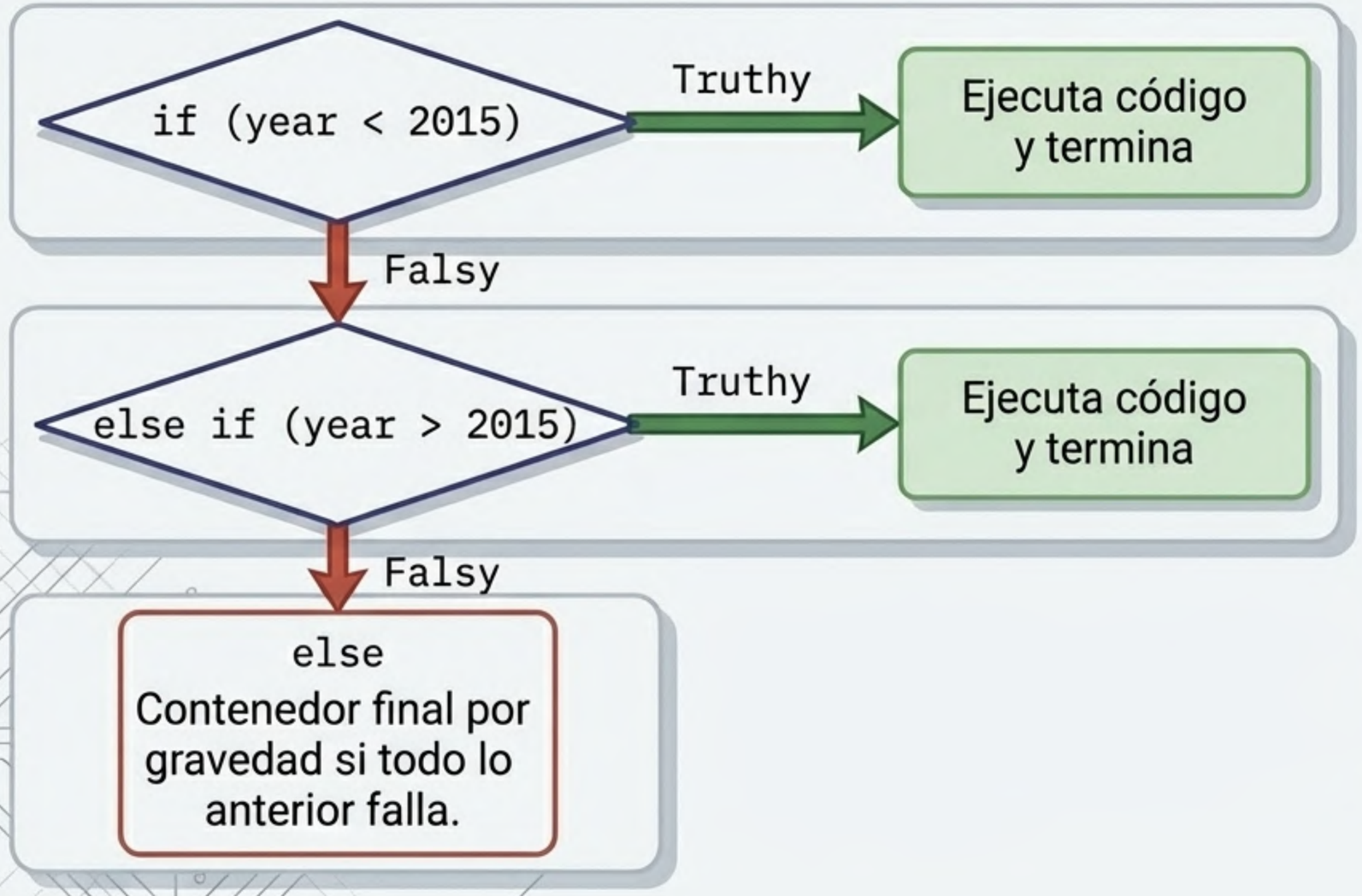
```
null > 0    // false
null == 0   // false
null >= 0   // TRUE
```

Los operadores relacionales (>=) fuerzan a `null` a convertirse en un número (0).

`0 >= 0` es matemáticamente cierto, aunque `null == 0` sea falso.

NotebookLM

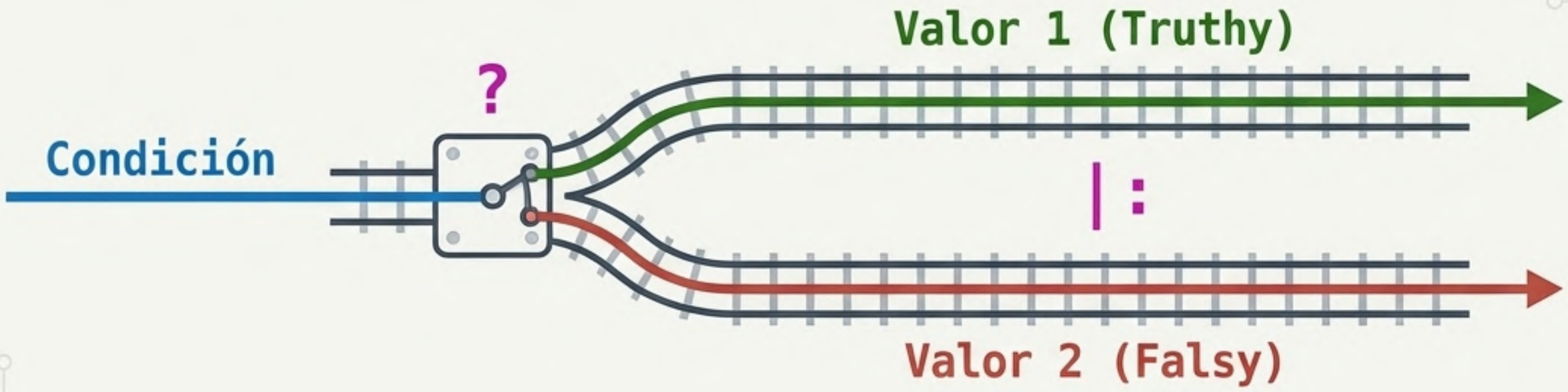
# Enrutamiento Secuencial: Bloques Condicionales




La estructura `if` evalúa su condición forzándola a Booleano. JavaScript detiene la evaluación entera en la primera condición **Truthy** que encuentra.

NotebookLM

# El Interruptor Horizontal: Operador Ternario (?:)




El único operador con tres operandos. Ideal para asignar rápidamente un valor u otro a una variable.

**Anti-patrón**

~~a ? b : c ? d : e ? f : g~~

Peligro: Nunca anides operadores ternarios. Sacrifica la legibilidad. Usa if/else en su lugar.

NotebookLM

# Los Guardianes Lógicos: Circuitos AND y OR

## Circuito **AND** (&&)

La energía solo pasa si **AMBOS** interruptores están cerrados.

## Circuito **OR** (||)

La energía pasa si **AL MENOS UNO** de los interruptores está cerrado.

## El Secreto de JS:

A diferencia de otros lenguajes, && y || no siempre devuelven `true` o `false` puro...

Devuelven el **VALOR REAL** del operando que detuvo o completó el circuito.

NotebookLM

# Evaluación de Cortocircuito (Short-Circuiting)

## && (El Buscador Falsy)

Escanea buscando el primer valor Falsy. Si lo encuentra, hace CORTOCIRCUITO, se detiene y devuelve ese valor exacto.

## || (El Buscador Truthy)

Escanea buscando el primer valor Truthy. Si lo encuentra, hace CORTOCIRCUITO, en Pornito, se detiene y devuelve ese valor puro (ideal para valores por defecto).

JavaScript optimiza el rendimiento deteniendo la evaluación lo antes posible. Entender esta maquinaria te permite diseñar rutas de flujo, no solo escribir código.

NotebookLM